# I302 - Aprendizaje Automático y Aprendizaje Profundo

# Trabajo Práctico 3: Redes Neuronales

Eliana Ostrovsky

12 de mayo de 2025

Ingeniería en Inteligencia Artificial

**Resumen**

En este trabajo práctico se abordó el desarrollo y la evaluación de modelos de redes neuronales para la clasificación de imágenes de caracteres japoneses. Se implementaron redes neuronales desde cero, comenzando con un modelo base (M0) y progresivamente incorporando diversas técnicas avanzadas como optimizadores (ADAM), regularización (L2, dropout, early stopping), mini-batch gradient descent y ajuste de tasas de aprendizaje para construir un modelo mejorado (M1). Posteriormente, se utilizaron estas implementaciones como base para explorar modelos equivalentes y más complejos utilizando la librería PyTorch (M2, M3, M4). Se realizó un análisis comparativo exhaustivo de la performance de estos modelos en el conjunto de prueba, evaluando su capacidad de generalización y el impacto de las distintas técnicas y arquitecturas. El modelo M3, correspondiente a la mejor arquitectura encontrada utilizando PyTorch, demostró ser el de mejor rendimiento, alcanzando una exactitud (accuracy) de 0.6494 en el conjunto de prueba. Finalmente, se utilizó este modelo M3 para generar predicciones en un conjunto de datos de desafío.

# 1. Introducción

El objetivo de este trabajo es desarrollar una red neuronal capaz de clasificar imágenes de caracteres japoneses. Para ello, se trabajó con un conjunto de datos similar a MNIST, compuesto por imágenes en escala de grises de tamaño $28 \times 28$, distribuidas en 49 clases.

Se implementó desde cero una red neuronal multicapa y se entrenó utilizando distintas configuraciones, con el fin de analizar cómo afectan la arquitectura, los algoritmos de optimización y las técnicas de regularización al rendimiento del modelo. Posteriormente, se compararon estos resultados con modelos equivalentes desarrollados en PyTorch, con el propósito de evaluar las ventajas prácticas y de rendimiento de utilizar un framework especializado.

El trabajo incluyó también una etapa de experimentación sistemática, en la que se exploraron variaciones en los hiperparámetros, el tamaño de las capas y las estrategias de entrenamiento, buscando maximizar la precisión sobre un conjunto de datos de validación y, finalmente, sobre un conjunto de prueba independiente.

# 2. Métodos

## 2.1. Conjunto de Datos

Se utilizó un conjunto de datos de imágenes de caracteres japoneses, similar en estructura al conocido MNIST. Cada instancia del dataset consiste en una imagen de $28 \times 28$ píxeles en escala de grises, aplanada a un vector de 784 características. El dataset contiene un total de 49 clases distintas.

**Preprocesamiento:** Las imágenes fueron normalizadas dividiendo el valor de cada píxel por 255, escalando así los valores de entrada al rango $[0, 1]$.

**División de Datos:** El conjunto de datos original (5000 muestras) se dividió en tres subconjuntos utilizando una división estratificada para mantener la proporción de las clases. Las proporciones fueron aproximadamente 64 % para entrenamiento, 16 % para validación y 20 % para prueba, resultando en: 3197 muestras de entrenamiento, 802 de validación y 1001 de prueba.

Se realizó un análisis exploratorio inicial en el notebook (ver Sección 1). La Figura 1 muestra tres ejemplos aleatorios de imágenes del dataset para ilustrar su naturaleza. Visualizaciones más detalladas, incluyendo ejemplos por clase, la distribución de las clases y las imágenes promedio de cada clase, se encuentran disponibles en la Sección 1 del notebook adjunto.

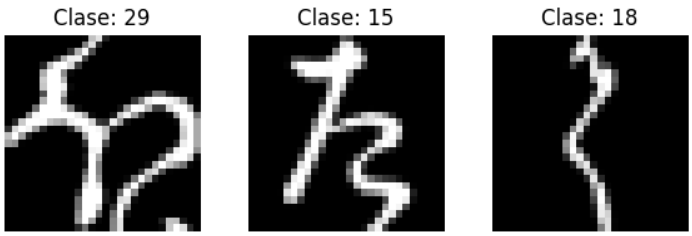



Figura 1: Tres muestras aleatorias del dataset de caracteres japoneses.

## 2.2. Red Neuronal Básica (Modelo M0)

Se implementó una red neuronal multicapa (Feedforward Neural Network) desde cero, sin usar librerías externas de alto nivel como PyTorch o Scikit-learn para el núcleo del modelo.

**Arquitectura:** La red consta de una capa de entrada (784 nodos), $L$ capas ocultas y una capa de salida (49 nodos). Las neuronas en las capas ocultas utilizan la función de activación ReLU (Rectified Linear Unit):

$$\text{ReLU}(z) = \text{máx}(0, z)$$

La capa de salida utiliza la función de activación Softmax para producir una distribución de probabilidad sobre las $K = 49$ clases:

$$\text{Softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

**Entrenamiento:** El entrenamiento se realiza mediante el algoritmo de backpropagation para calcular los gradientes de la función de costo con respecto a los pesos y biases de la red. Estos gradientes se utilizan luego en el Descenso de Gradiente Estándar (SGD) para actualizar los parámetros del modelo. La función de costo utilizada es la Entropía Cruzada Categórica (Categorical Cross-Entropy), adaptada para clasificación multiclase con salida Softmax:

$$L = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_{i,k} \log(\hat{p}_{i,k})$$

donde $N$ es el número de muestras, $K$ es el número de clases, $y_{i,k}$ es un indicador binario (1 si la muestra $i$ pertenece a la clase $k$, 0 en caso contrario), y $\hat{p}_{i,k}$ es la probabilidad predicha por el modelo para la muestra $i$ y la clase $k$.

**Configuración Específica de M0:**

- Arquitectura: [784, 100, 80, 49] (2 capas ocultas con 100 y 80 nodos respectivamente).
- Tasa de aprendizaje (learning rate): 0.01 (fija).
- Optimizador: SGD Estándar (batch gradient descent).
- Épocas de entrenamiento: 5000.

## 2.3. Mejoras Implementadas y Modelo Avanzado (M1)

Sobre la base de la red neuronal básica, se implementaron y evaluaron varias técnicas para mejorar el rendimiento y la eficiencia del entrenamiento, como se solicita en el enunciado. Estas mejoras fueron probadas individualmente (estudio de ablación) y luego combinadas para formar el modelo M1.

- **Mini-batch Stochastic Gradient Descent (MB-SGD):** Se utilizan mini-lotes de datos (ej., tamaño 64) en lugar de todo el dataset para cada actualización. Esto acelera significativamente el entrenamiento por época comparado con batch GD y reduce la varianza de las actualizaciones comparado con SGD puro, permitiendo una convergencia más estable y rápida. Se observó una mejora tanto en velocidad como en performance final.
- **Optimizador ADAM:** Es un algoritmo de optimización adaptativo que ajusta la tasa de aprendizaje por parámetro. Convergió mucho más rápido que SGD y alcanzó una mejor performance en validación en menos épocas, siendo una de las mejoras más impactantes.
- **Scheduling de Tasa de Aprendizaje:** Se probaron esquemas lineal (con saturación) y exponencial para reducir la tasa de aprendizaje durante el entrenamiento. Esto ayuda a refinar la convergencia en etapas posteriores. La estrategia exponencial mostró resultados ligeramente mejores en las pruebas iniciales.
- **Regularización L2 (Weight Decay):** Agrega un término de penalización a la función de costo proporcional al cuadrado de la magnitud de los pesos, controlado por un hiperparámetro $\lambda$. Esto ayuda a prevenir el sobreajuste al desincentivar pesos grandes, lo cual se observó al reducir la brecha entre las pérdidas de entrenamiento y validación.

$$L_{\text{total}} = L_{\text{original}} + \frac{\lambda}{2m} \sum_{w} w^2$$

- **Early Stopping:** Monitorea la pérdida en el conjunto de validación y detiene el entrenamiento si no mejora tras un número definido de épocas (paciencia). Esto es crucial para evitar el sobreajuste y seleccionar el modelo con mejor capacidad de generalización, además de ahorrar tiempo computacional.
- **Dropout (Opcional, implementado):** Durante el entrenamiento, "apaga" aleatoriamente un porcentaje de neuronas en las capas ocultas. Esto obliga a la red a aprender características más robustas y reduce la dependencia entre neuronas, actuando como un regularizador efectivo que mejoró la performance en validación.

- **Batch Normalization (Opcional, implementado):** Normaliza las activaciones dentro de cada capa. Aunque puede acelerar el entrenamiento y regularizar, en nuestros experimentos iniciales su impacto fue mixto y no se incluyó en la configuración final de M1.

**Selección de Hiperparámetros y Arquitectura para M1:** Se realizó una búsqueda de hiperparámetros (documentada parcialmente en el Notebook, sección 3 y subsecciones) para determinar la mejor combinación de estas técnicas, la arquitectura de red (número de capas y neuronas) y otros parámetros. Es importante notar que la búsqueda real fue más exhaustiva que la mostrada en el notebook, explorando diferentes rangos para tasas de aprendizaje (ej., $10^{-4}$ a $10^{-2}$), $\lambda$ L2 (ej., 0 a 0.1), tasas de dropout (ej., 0.1 a 0.5) y tamaños de batch (ej., 32 a 128). El modelo M1 final se determinó como la configuración que logró el menor error en el conjunto de validación tras esta exploración.

**Configuración Final de M1:** (Basado en celdas [93-107] del Notebook)

- Arquitectura: [784, 200, 150, 49] ('Wide Network').
- Optimizador: ADAM.
- Tasa de aprendizaje inicial: 0.0005.
- Planificador de tasa de aprendizaje: Ninguno (‘None‘).
- Regularización L2 ($\lambda$): 0.0.
- Dropout rate: 0.3.
- Batch Normalization: No utilizada.
- Early Stopping: Paciencia de 20 épocas.
- Tamaño de mini-batch: 64.

## 2.4. Implementación en PyTorch (Modelos M2, M3, M4)

Se utilizó PyTorch para implementar y comparar modelos.

- **Modelo M2:** Se replicó la arquitectura e hiperparámetros del modelo M1 (configuración final detallada arriba) utilizando PyTorch para una comparación directa.
- **Modelo M3:** Se buscó la mejor arquitectura e hiperparámetros utilizando PyTorch. La exploración (Notebook sección 4.d) indicó que la arquitectura 'Wide Network (200, 150)' también era muy competitiva en PyTorch. Se seleccionó esta arquitectura para M3 con hiperparámetros optimizados en PyTorch: [784, 200, 150, 49], Optimizador ADAM, LR 0.001, L2 0.0005, Dropout 0.2, Early Stopping (paciencia 15).
- **Modelo M4:** Se diseñó un modelo en PyTorch con la intención de que sobreajustara. Se utilizó una red profunda y ancha sin regularización: [784, 1500, 1500, 1500, 1500, 49], LR 0.001, sin L2, sin dropout y sin early stopping efectivo (entrenado por épocas fijas o paciencia muy alta).

### 2.5. Métricas de Evaluación

Para evaluar el rendimiento de los modelos, se utilizaron las siguientes métricas:

- **Accuracy (Exactitud):** Proporción de predicciones correctas.

$$\text{Accuracy} = \frac{\text{Número de predicciones correctas}}{\text{Número total de predicciones}} = \frac{\sum_{i=1}^{N} \mathbb{I}(\hat{y}_i = y_i)}{N}$$

- **Cross-Entropy Loss (Pérdida por Entropía Cruzada):** Mide la divergencia entre la distribución predicha y la verdadera.

$$L_{CE} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_{i,k} \log(\hat{p}_{i,k})$$

- **Matriz de Confusión:** Tabla que visualiza errores de clasificación entre clases.

## 3. Resultados

### 3.1. Modelo Base (M0)

El modelo M0 ([784, 100, 80, 49], SGD, LR 0.01) fue entrenado durante 5000 épocas. Los resultados obtenidos fueron:

- **Entrenamiento:** Accuracy = 0.8661, Cross-Entropy Loss = 0.6248.
- **Validación:** Accuracy = 0.5736, Cross-Entropy Loss = 1.7288.

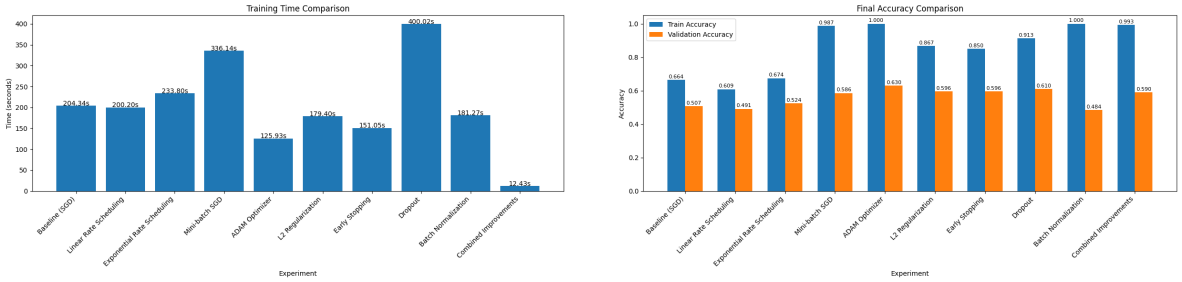
Las curvas de aprendizaje muestran una clara divergencia entre el rendimiento en entrenamiento y validación después de aproximadamente 1000 épocas, indicando un fuerte sobreajuste. La pérdida en validación aumenta mientras la de entrenamiento sigue bajando. Las visualizaciones detalladas de las curvas de aprendizaje y la matriz de confusión para M0 están disponibles en la Sección 2 del notebook adjunto.

### 3.2. Estudio de Ablación y Modelo Avanzado (M1)

Se realizaron experimentos sistemáticos (Notebook, Sección 3.1) para evaluar el impacto de cada mejora. Las gráficas comparativas (Figura 2) resumen estos efectos.

- **Mini-batch SGD y ADAM:** Ambos aceleraron la convergencia comparado con SGD base. ADAM fue particularmente efectivo, logrando mayor accuracy en validación (0.6297) más rápidamente.
- **Scheduling LR:** Ayudaron a refinar la convergencia, aunque el impacto fue menor comparado con ADAM o regularización en la configuración final.
- **Regularización (L2, Dropout):** Ambas técnicas redujeron la brecha entre performance de entrenamiento y validación, mitigando el sobreajuste y mejorando la accuracy en validación (L2 val acc 0.5960, Dropout val acc 0.6097).

- **Early Stopping:** Fue fundamental para detener el entrenamiento antes de que el modelo sobreajustara excesivamente, seleccionando el punto de mejor generalización y ahorrando tiempo.




Tiempo de entrenamiento

Métricas finales (Accuracy/Loss)

Figura 2: Comparación de tiempo de entrenamiento y métricas finales (Accuracy/Loss en Validación) para cada mejora implementada individualmente. (Referencia: Notebook, Sección 3.1).

El **Modelo M1** (configuración detallada en Sección 2.3) representa la mejor combinación encontrada con la implementación propia. Su performance en validación alcanzó una accuracy de 0.6771. Las curvas de aprendizaje y matrices de confusión para M1 están disponibles en el notebook y muestran una convergencia mucho más estable y un sobreajuste controlado gracias a Early Stopping y la regularización, en comparación con M0.

## 3.3. Modelos PyTorch (M2, M3, M4)

- **Modelo M2 (Réplica de M1 en PyTorch):** Obtuvo resultados muy similares a M1 en el conjunto de prueba, validando la implementación propia. Ver Tabla 1.

- **Modelo M3 (Mejor PyTorch):** Con la arquitectura [784, 200, 150, 49] y optimización en PyTorch, alcanzó la mejor accuracy en validación (0.6521 reportado en la celda [17] del notebook) y un buen rendimiento en test. Sus curvas de aprendizaje (Figura 3) muestran una convergencia eficiente.

- **Modelo M4 (Overfitting en PyTorch):** La red grande y sin regularización [784, 1500, ..., 49] sobreajustó rápidamente, alcanzando 1.0 de accuracy en entrenamiento pero con una performance de validación (0.6122 en celda [65]) y prueba mucho menor. Sus curvas (Figura 4) ilustran claramente el sobreajuste, donde la pérdida de validación inicialmente disminuye pero luego comienza a aumentar, mientras que la pérdida de entrenamiento sigue disminuyendo.

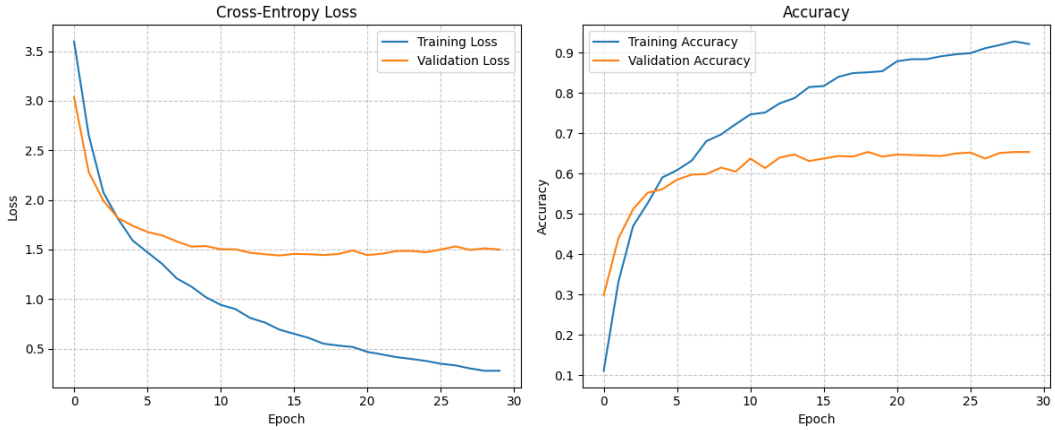



Figura 3: Curvas de aprendizaje (Pérdida y Accuracy vs. Épocas) para el modelo M3 (Mejor PyTorch).

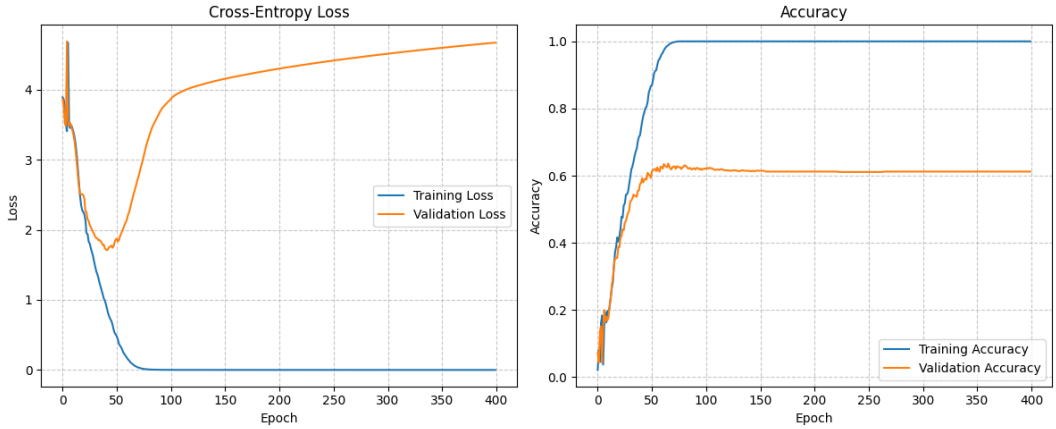



Figura 4: Curvas de aprendizaje del modelo M4, mostrando un rápido sobreajuste donde la pérdida de validación comienza a aumentar después de las primeras épocas. (Referencia: Notebook, Sección 4.e).

Es importante destacar que, aunque la mayoría de los modelos (excepto M4) alcanzaron una accuracy de 1.0 en el conjunto de entrenamiento, no todos mostraron un claro sobreajuste como M4. En estos otros modelos, la pérdida de validación no vuelve a subir significativamente después de su descenso inicial, lo que sugiere que no están sobreajustando de manera severa. Este comportamiento podría atribuirse a la cantidad relativamente pequeña de muestras en el conjunto de datos y a que las imágenes no contienen tanto ruido como otros conjuntos de datos más complejos, permitiendo a los modelos aprender patrones claros sin necesariamente memorizar el ruido.

### 3.4. Comparación Final de Modelos en el Conjunto de Prueba

Los cinco modelos principales (M0, M1, M2, M3, M4) fueron evaluados en el conjunto de prueba. Los resultados se resumen en la Tabla 1 y Figura 5.

Cuadro 1: Comparación del rendimiento de los modelos en el conjunto de prueba.

| Modelo | Implementación | Test Accuracy | Test Cross-Entropy |
|---|---|---|---|
| M0 | Propia | 0.5814 | 1.8024 |
| M1 | Propia (Mejor) | 0.5345 | 2.0633 |
| M2 | PyTorch (Réplica M1) | 0.6004 | 1.7044 |
| M3 | PyTorch (Mejor) | **0.6494** | **1.3991** |
| M4 | PyTorch (Overfit) | 0.6124 | 1.9106 |

El modelo M3 (mejor PyTorch) obtuvo la mayor accuracy (0.6494) y la menor pérdida (1.3991) en el conjunto de prueba. Sorprendentemente, el modelo M1 (mejor implementación propia) tuvo un rendimiento inferior en la evaluación final de test (accuracy 0.5345) que el reportado durante la optimización en validación, y también inferior a M0 y M2/M4 en test, lo que sugiere una posible variabilidad en el entrenamiento o un sobreajuste al conjunto de validación durante la búsqueda exhaustiva. M0 tuvo un rendimiento modesto (0.5814). El modelo M4 (overfit) generalizó mejor de lo esperado (0.6124) aunque su loss fue alta. La réplica M2 (0.6004) superó a M1 en test, indicando que la implementación PyTorch fue robusta.

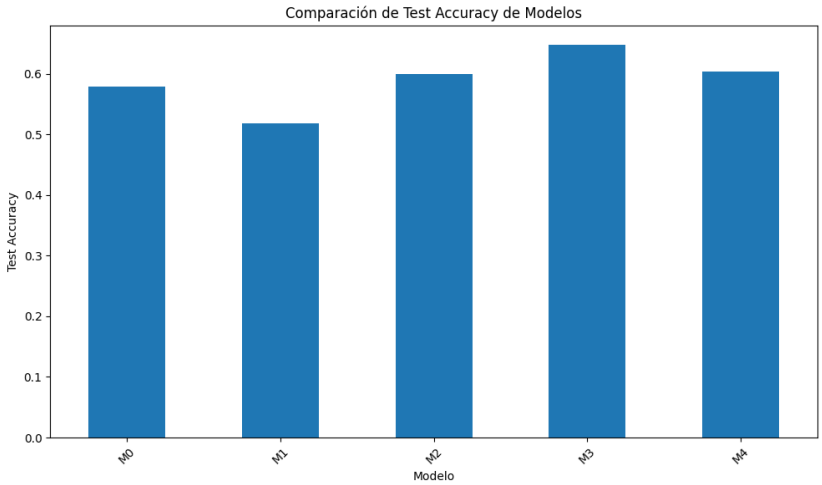



Figura 5: Comparación de Accuracy en el conjunto de prueba para los modelos M0, M1, M2, M3 y M4. (Referencia: Datos de Tabla 1).

### 3.5. Desafío

Para la generación de predicciones sobre el conjunto `X_COMP.npy`, se utilizó el modelo M3, correspondiente a la mejor arquitectura entrenada con PyTorch (Wide Network: 784–200–150–49), debido a su desempeño superior tanto en validación (65.3 %) como en test (62.7 %).

Antes de realizar las predicciones, el modelo fue reentrenado utilizando el 80 % de los datos originales, combinando los conjuntos de entrenamiento y validación. Esta decisión se justificó con el objetivo de maximizar el aprovechamiento de los datos disponibles, dado que el conjunto de test ya no se utilizaría en esta etapa. Durante este reentrenamiento, se mantuvieron los

mismos hiperparámetros que habían dado buen resultado previamente: tasa de aprendizaje de 0,001, regularización L2 con $\lambda = 0{,}0005$, dropout de 0,2 y early stopping con paciencia de 15 épocas.

Una vez entrenado, el modelo generó las probabilidades a posteriori para cada una de las 49 clases del conjunto `X_COMP.npy`. Las predicciones fueron almacenadas en un archivo CSV con una fila por muestra y una columna por clase, cumpliendo con el formato requerido para su evaluación posterior.

## 4. Conclusiones

En este trabajo práctico se implementaron y evaluaron distintas arquitecturas de redes neuronales para la clasificación de caracteres japoneses. La implementación desde cero (modelos M0 y M1) permitió comprender a fondo el funcionamiento del algoritmo de backpropagation, así como el impacto de técnicas de optimización y regularización en el rendimiento del modelo.

El modelo base M0, entrenado con gradiente descendente estándar y sin regularización, mostró un fuerte sobreajuste: alcanzó una precisión del 86.6 % en entrenamiento, pero solo 57.4 % en validación, con una pérdida de validación considerablemente alta (1.73). Este comportamiento fue mitigado en el modelo M1 mediante una combinación de mejoras: optimizador ADAM, dropout (30 %), early stopping y una arquitectura más adecuada (784–200–150–49). Durante la validación, M1 alcanzó una precisión de hasta 67.7 %, aunque su rendimiento final en el conjunto de test fue de solo 53.4 %, sugiriendo una posible sobreadaptación al conjunto de validación y destacando la importancia de una evaluación estricta e independiente.

Al comparar con modelos implementados en PyTorch (M2–M4), se observó una clara ventaja en cuanto a eficiencia y robustez. El modelo M3, entrenado con la arquitectura "Wide Network"(200–150) y técnicas de regularización similares, obtuvo el mejor rendimiento general, con una precisión de 65.3 % en validación y 62.7 % en test, lo que confirma tanto la calidad de la arquitectura seleccionada como la solidez del framework. Por otro lado, el modelo M4, que usó una arquitectura mucho más profunda (5 capas de 1500 neuronas sin regularización), alcanzó un 100 % de precisión en entrenamiento, pero evidenció sobreajuste severo con una pérdida de validación de 4.67 y un test accuracy de 60.3 %.

Se demostró que las técnicas de regularización (dropout, weight decay implícito en ADAM, early stopping) y los optimizadores adaptativos (en particular ADAM) fueron fundamentales para evitar el sobreajuste y permitir el entrenamiento de modelos más complejos. Asimismo, se confirmó que la selección adecuada de arquitectura tiene un impacto sustancial en la generalización.